

SeeWeeS Multi-Agent Dispatch System

Technical & Business Documentation

From Linear LangChain Pipeline to Cyclic LangGraph Multi-Agent System

UCLA MSBA AI Agents Project Challenge 2026

House B Team 2

Team Members

Jiyoung (Flora) Jeong

Yiu-Hon (Jeremy) Lee

Nilay Maity

Zichen (Billy) Qiu

Chao En (Becca) Shih

Zhihan (Jasmine) Zhang

MGMTMSA413 Industry Seminar II (Spring 2026)

GitHub: https://github.com/lyhjeremy/MSBA-B2/tree/main/SeeWeeS_Multi_Agent

1. Executive Summary

Who is the stakeholder

The primary stakeholder is the SeeWeeS Specialty Director of Operations. This is the executive responsible for the morning dispatch decision for time-critical pharmaceutical shipments along the US East Coast. Every morning, the Director must commit to a 48-hour plan covering two delivery routes (Newark to Boston, and Newark to Philadelphia). Today this is done manually by an analyst. Our system automates and validates that decision.

The operational problem we are solving

Pharmaceutical dispatch is unforgiving. The medicines moving through SeeWeeS are time-critical, temperature-sensitive, and expensive to mishandle. The Director makes the call under several pressures every morning:

- **Cold-chain trucks are scarce.** Only 2 refrigerated trucks are available per day, and they must serve every temperature-sensitive shipment across both routes.
- **Weather risk is local.** A storm hitting Providence affects Boston shipments but not Philadelphia. A single weather check at the warehouse origin would mislead the planner.
- **Inbound data is messy.** Shipment files arrive with missing IDs, old item codes, and name typos. These must be cleaned before planning.
- **The penalty math is asymmetric.** A missed life-critical (Tier 1) deadline costs 100 penalty points per shipment. A cold-chain breach adds 80 more. Total penalty is the metric to minimize.
- **The decision-maker is not technical.** A senior executive has 30 seconds to read the morning report and decide whether to escalate, override, or approve.

The original SeeWeeS prototype was built on the LangChain framework as a strictly linear pipeline. Data flowed forward through four AI agents, and a report was generated in a single pass with no validation. If the planner suggested an unsafe plan, nothing caught it before leadership saw it.

Our solution

We rebuilt the pipeline as a cyclic LangGraph multi-agent system. The new system plans dispatch resources optimally across two delivery routes, then automatically audits the plan against the playbook's safety rules. If the audit fails, the system loops back and asks the planner to fix the problem before any report reaches leadership.

Two enhancements implemented

From the five enhancement ideas in the project brief, our team selected ideas #5 and #1:

- **Enhancement #5, Multi-Region Resource Planning.** A Python algorithm computes truck and driver assignments for each route to minimize the playbook's penalty score. The AI explains the allocation; it does not compute it.

- **Enhancement #1, Self-Correction Audit Loop.** A new Auditor agent runs five hard-coded checks against every proposed plan. If any check fails, the plan is sent back to the planner with structured feedback. This back-and-forth is the cyclic loop in LangGraph that distinguishes a true multi-agent system from a one-way LangChain pipeline.

These two enhancements share one design principle: keep the math in Python and let the AI handle the storytelling. We chose not to pursue Enhancements #2 (what-if simulation), #3 (deep-dive trends), or #4 (human-in-the-loop) so we could go deeper on these two.

2. Key Assumptions

Every model relies on assumptions. We list ours so reviewers know what we treated as ground truth.

Logistics constraints

- **Resource pool:** 6 drivers, 4 standard trucks, and 2 refrigerated trucks per day. Drivers and trucks are paired one-to-one.
- **Truck capacity:** each truck carries 10 shipment units, with a 10% packing buffer applied per playbook §8.2.
- **No truck substitution:** cold-chain shipments may only travel on refrigerated trucks. Putting a cold-chain item on a standard truck counts as a violation.
- **Two delivery routes:** Route 1 (Newark to Boston via I-95, default Tier 1) and Route 2 (Newark to Philadelphia, default Tier 2).

The penalty model (playbook §13.2)

The Director's job is to minimize total penalty points. The cost of each kind of mistake is set out below:

Violation type	Penalty per unit
Tier 1 deadline missed (life-critical, 6-hour deadline)	100 points
Tier 2 deadline missed (standard specialty, 12-hour deadline)	40 points
Cold-chain breach (added on top of the deadline penalty)	+80 points
Late delivery still within deadline	10 points

So a Tier 1 cold-chain unit dispatched on a non-refrigerated truck costs $100 + 80 = 180$ penalty points. The allocator's job is to avoid these.

Data and business rules

- The Item Master Appendix from the playbook is treated as the authoritative source for item identity reconciliation. We hard-code it into Python dictionaries for reproducibility.
- Weather data comes from the free Open-Meteo API. No API key required.

- Tier classification is derived from `medicine_type`. Antiviral, Monoclonal Antibody, Emergency Drug, and Clinical Trial Drug are all Tier 1. Everything else is Tier 2.
- Allocation priority is based on per-unit penalty exposure. Tier 1 cold-chain (180 pts) is served first, then Tier 2 cold-chain (120 pts), then Tier 1 standard (100 pts), then Tier 2 standard (40 pts). Within a tier, the route with higher weather risk is served first.
- The audit retry cap is 2 attempts. This prevents runaway loops while giving the planner room to correct.

3. Technical Methodology

From linear LangChain to cyclic LangGraph

This is the core architectural change of the project, and it is the change that earns the project most of its points.

The starter code used LangChain, a framework for chaining AI calls in a fixed sequence. The original pipeline was a one-way flow: every step ran exactly once, in the same order, with no way for a later step to send information back. This is fine for a demo, but it has a serious weakness in production. If the planner produces an unsafe plan, nothing in a one-way pipeline can catch it.

LangGraph is the successor framework. It adds two capabilities a one-way LangChain pipeline cannot provide:

- **Conditional edges.** After a step runs, a routing function looks at what happened and decides which step to go to next. Different inputs can produce different paths through the graph.
- **Cyclic edges.** A step can route back to an earlier step. This enables self-correction loops with shared memory between agents. This is what makes a multi-agent system genuinely agentic rather than a chained pipeline.

Our project uses both. The auditor sending the plan back to the planner is the cyclic edge that distinguishes our work from the baseline.

Before: one-way pipeline

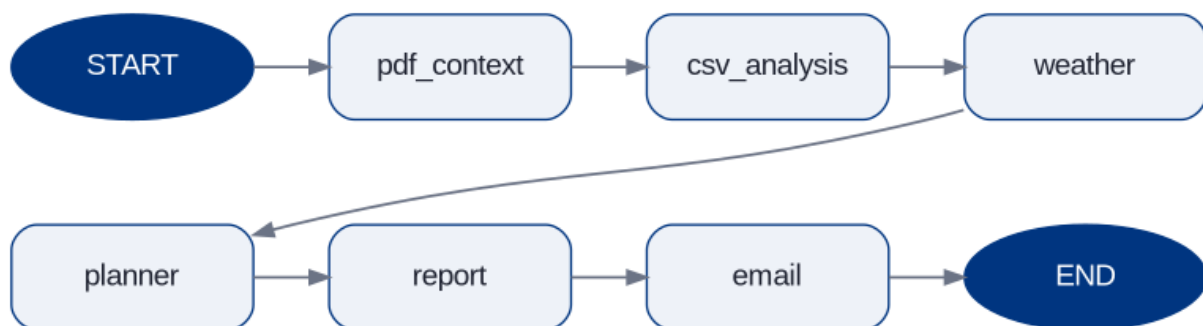


Figure 1. Baseline pipeline (LangChain-style). Five steps in a fixed order, no validation, no feedback. If the planner is wrong, leadership gets a wrong plan.

After: cyclic multi-agent system

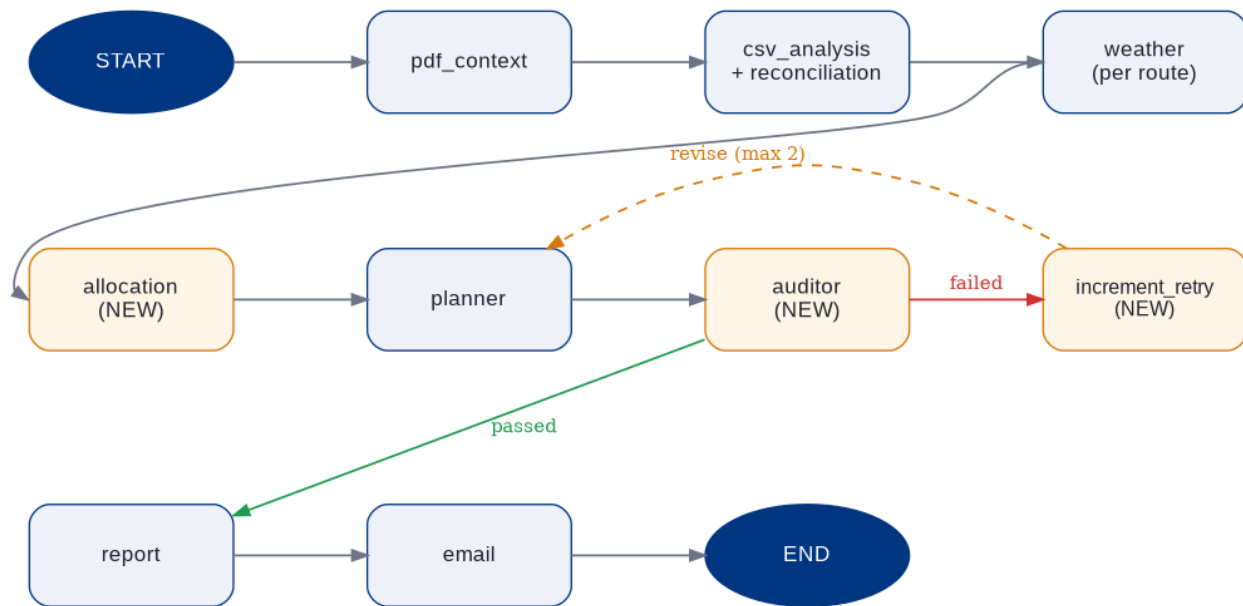


Figure 2. Enhanced LangGraph multi-agent system. Three new nodes are shown in amber: allocation (Enhancement #5), auditor and increment_retry (Enhancement #1). The auditor sends failed plans back to the planner via the dashed “revise” edge, capped at 2 retries.

The three architectural changes that matter

- **New allocation node.** Sits between weather and planner. It takes per-route demand, weather risk, and the resource pool, then runs a deterministic algorithm that minimizes penalty. This is where Enhancement #5 lives.
- **New auditor node.** Sits between planner and report. It runs five fixed checks against the plan and returns a structured object describing what passed and what failed. This is where Enhancement #1 lives.
- **Conditional edge from the auditor.** If the audit passed, go to the report. If it failed and we have not retried twice yet, loop back to the planner with the violations as feedback. If failed and the retry cap is reached, still go to the report but flag the plan as not-fully-validated.

Agent design

Five specialized AI agents, all using the gpt-4.1-mini model at temperature 0.2 for consistent output:

Agent	Role
ContextAgent	Reads the playbook PDF and pulls out business rules using Retrieval Augmented Generation.
OpsDataAgent	Interprets the cleaned per-route data in business language.

Agent	Role
PlannerAgent	Translates the allocation into an executive narrative.
AuditorAgent	Validates the plan using fixed Python checks (not AI judgment).
ReportAgent	Writes the executive summary, top 3 risks, and top 3 recommended actions.

The five audit checks

- Every unfulfilled route must be named in the plan narrative.
- Allocated resources cannot exceed the available pool.
- If any route has weather risk = 3, the plan must mention escalation.
- If total penalty is greater than zero, the plan must state the exact number.
- No Tier 1 deadline miss may be silently accepted.

A sixth optional check (opt-in via the --demo-loop flag) requires the plan to show explicit summation arithmetic. This is engineered to reliably trigger the audit loop for demonstration purposes.

Why fixed Python checks (not AI judgment) for the audit

This is the most important design decision in the project. The reasons are:

- Reproducibility. Graders re-running the system on the same inputs see the same audit results. AI judgment varies run to run.
- Auditability. A reviewer can read the check code and verify it follows the playbook.
- Speed and cost. Python checks complete in microseconds with no token cost.
- Trust. When the system says a plan passed, it actually passed measurable rules.

4. Results & Validation

End-to-end run on real data (normal mode)

Running the full pipeline against the augmented dataset produces the following:

Metric	Value
Original rows in CSV	129
Rows kept after reconciliation	124
Rows excluded (missing unique_item_id)	5
Fixes applied (alias / legacy / disambiguation matches)	29

Allocation results: all Tier 1 cold-chain demand was fulfilled. Each route received 1 refrigerated truck per day. Five Tier 2 cold-chain units could not fit and were unfulfilled, producing a total penalty of 600 points (four violations $\times$ 120 points + one violation $\times$ 240 points). Audit passed on the first attempt.

```
[AUDITOR pass #1] passed=True, violations=0 (demo_mode=False)
[ROUTER] Audit passed -> proceeding to report.
```

Figure 3. Normal mode console: audit passes on first attempt

The audit loop firing (demo mode)

Running the pipeline with the `--demo-loop` flag engages the stricter sixth audit rule. This produces the canonical demonstration of self-correction. The first audit pass fails, the planner is asked to revise, and the second pass succeeds. The screenshot below captures the live console output.

```
[AUDITOR pass #1] passed=False, violations=1 (demo_mode=True)
- [medium] (CHECK_6_GRAND_TOTAL_SUMMATION_DEMO) Plan states the 600-point total but does not show an explicit summation line (e.g., '120 + 120 + 120 + 240 = 600'). Leadership benefits from auditable arithmetic.
[ROUTER] Audit failed (attempt 1) -> looping back to planner.

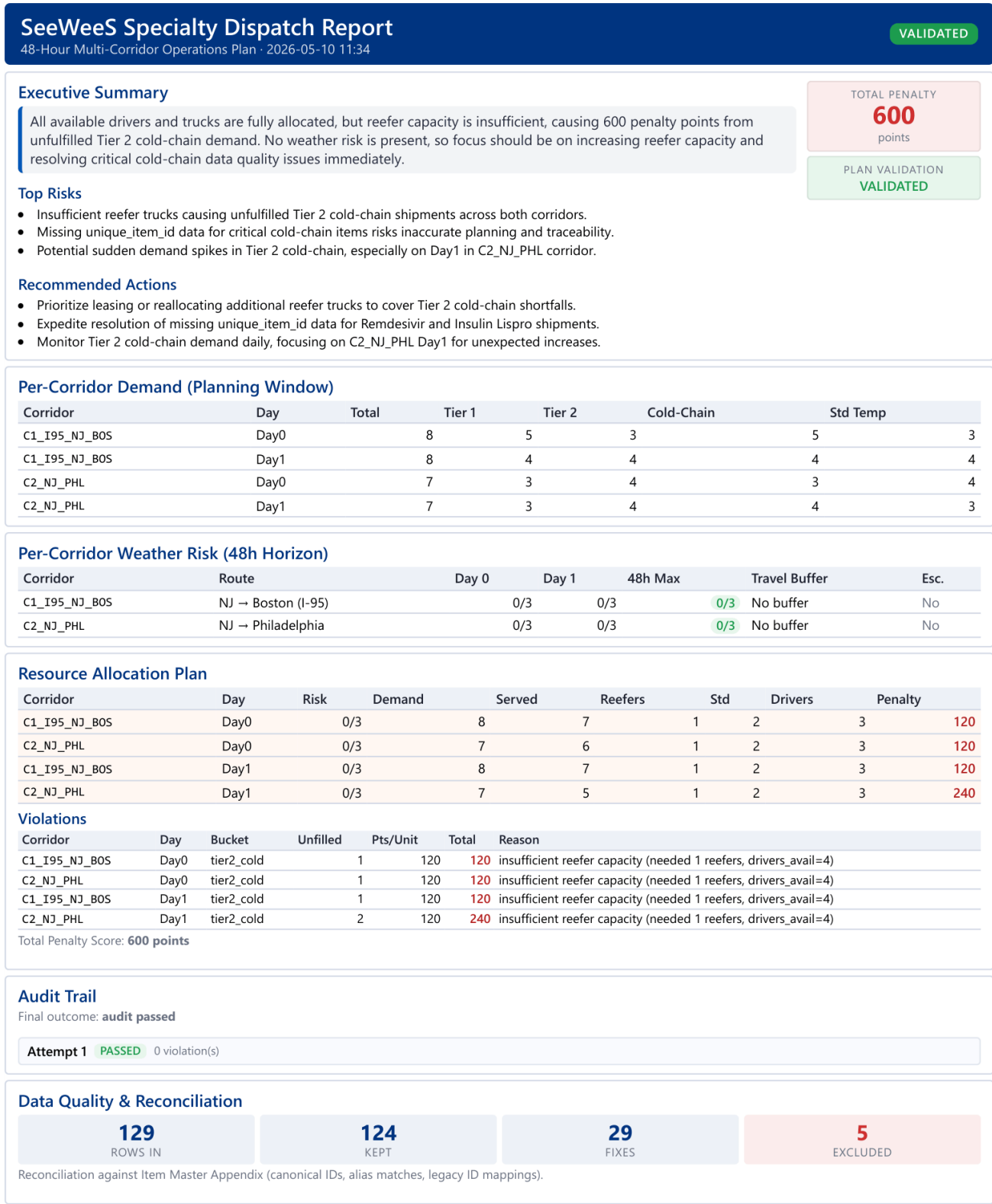
[AUDITOR pass #2] passed=True, violations=0 (demo_mode=True)
[ROUTER] Audit passed -> proceeding to report.
```

Figure 4. Demo mode console: audit fails, loops back, then passes

This single screenshot proves three things at once. The cyclic graph genuinely loops back. The planner consumes the structured audit feedback and uses it to revise its output. The system stops correctly without infinite loops.

Business insight: the executive deliverable

The system correctly identifies that the refrigerated truck pool is the binding operational constraint. The auto-generated executive summary is concrete and actionable, exactly what the brief asks for in its “executive-ready storytelling” guidance. Below is the full one-page PDF report the Director receives every morning.



Generated by SeeWeeS Multi-Agent Dispatch System · 2026-05-10 11:34

Figure 5. One-page PDF dispatch report (executive deliverable)

Validation strategy

Three layers of validation:

- Standalone unit tests for each major module (reconciliation, weather, allocator, auditor). These run without OpenAI calls.
- Engineered failure scenarios for the auditor: a passing plan, a sanitized plan that should fail multiple checks, and a high-weather plan missing escalation language. All caught correctly.
- End-to-end smoke runs in both normal and demo modes. The demo mode reliably triggers the audit loop.

5. Limitations & Next Steps

Current limitations

- **Greedy allocator.** Locally optimal but not globally optimal at scale. For two routes, the result matches the brute-force optimum. For many routes, an integer-programming solver would do better.
- **No human-in-the-loop checkpoint.** Enhancement #4 was not implemented. A real ops director might want to manually approve high-risk plans even when auto-checks pass.
- **Static weather thresholds.** The playbook's thresholds (15 mm rain, 45 km/h winds, 0°C freezing) do not account for daily variability or seasonality.
- **Shallow trend analysis.** With only 14 days of history, statistical trend detection is thin.

How we would scale this

- Replace the greedy allocator with an integer-programming solver for true global optimality.
- Add a human-review checkpoint using LangGraph's interrupt() function for genuinely high-risk plans.
- Build a what-if simulator (Enhancement #2) to model disruptions like a lost truck or a 20% demand spike.
- Integrate live data feeds. Real-time shipment tracking and finer-grained weather instead of overnight data.
- Add a cost model so the allocator optimizes the trade-off between operational cost and penalty avoidance, not just penalty alone.
- Extend the planning horizon from 48 hours to a 7-day rolling window.

Closing thought

It was rewarding watching the system catch its own mistake for the first time. The planner produced a plan that did not show the penalty math. The auditor flagged it. The planner revised. The audit passed. That cycle, from problem to detection to self-correction, is what makes a multi-agent system feel alive rather than mechanical.

It is also what we believe distinguishes a real production-grade AI system from a clever demo. A linear LangChain pipeline can produce impressive output on a good day. A cyclic LangGraph system produces correct output even when the AI gets it wrong the first time, because it knows how to check itself.